INLP A2 Report

Maryala Sai Teja , 2024121007

March 5, 2026

1 Training Word Vectors

1.1 SVD

In this section, we describe the construction of word embeddings using a count-based approach followed by Singular Value Decomposition (SVD). The overall pipeline consists of (i) building a word–context co-occurrence matrix from the Brown Corpus, (ii) applying a weighting scheme to normalize raw counts, and (iii) performing truncated SVD to obtain dense low-dimensional word representations.

Context Window Size. We use a symmetric context window of size $w = 2$, meaning that for each target word, two words to the left and two words to the right are considered as context (excluding the target itself). A smaller window size emphasizes syntactic and local relationships between words, capturing short-range dependencies. Larger windows tend to capture broader semantic associations but may introduce noise from loosely related words. Since the Brown Corpus is relatively small, a window size of 2 provides a balance between meaningful contextual information and noise reduction.

Vocabulary Size. The vocabulary is restricted to the top 20,000 most frequent words in the corpus. This reduces sparsity and computational cost while retaining the majority of meaningful lexical items. Rare words tend to produce unreliable co-occurrence statistics and can negatively impact the quality of matrix factorization. I have tried using 10k which didn't include the required words which are required in the next section, I tried with the whole vocab but it was occupying a lot of memeory and it is getting killed.So I ended up with 20K

Weighting Scheme (PPMI). Raw co-occurrence counts are transformed using Positive Pointwise Mutual Information (PPMI). PMI measures how much more often two words co-occur than expected under statistical independence:

$$PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)}.$$

Since negative PMI values correspond to weak or uninformative associations, we use the positive variant:

$$PPMI(w, c) = \max(PMI(w, c), 0).$$

PPMI reduces the dominance of high-frequency words (e.g., stopwords) and highlights statistically significant word–context relationships. This normalization improves the semantic quality of embeddings when applying matrix factorization.

Embedding Dimensionality. We select an embedding dimension of $d = 100$. Truncated SVD computes the best rank-d approximation of the PPMI matrix, capturing the most informative latent structure while discarding noise. A dimensionality of 100 provides a good trade-off between representational capacity and computational efficiency. Lower dimensions may underfit semantic structure, while higher dimensions increase computational cost and risk overfitting sparse co-occurrence patterns(as the dataset is not that big).

Truncated SVD. Instead of computing full SVD, I used truncated SVD to extract only the top d singular components. This significantly reduces computational complexity and produces dense word vectors of size 100. According to the Eckart–Young theorem, truncated SVD provides the optimal low-rank approximation of the matrix under the Frobenius norm, making it well-suited for dimensionality reduction in distributional semantics.

1.2 Neural Word Embedding (CBOW)

I implemented the Continuous Bag of Words (CBOW) model with negative sampling to learn dense word representations from the Brown corpus. In the CBOW architecture, the model predicts a target word given its surrounding context words. The context representation is obtained by averaging the embedding vectors of words within a fixed window around the target.

Model Architecture

Let v_c denote the averaged context vector and v_t denote the embedding of the target word. The objective function with negative sampling is defined as:

$$\mathcal{L} = \log \sigma(v_t^\top v_c) + \sum_{k=1}^{K} \log \sigma(-v_{n_k}^\top v_c)$$

where:

- $\sigma(\cdot)$ is the sigmoid function,
- v_{n_k} are embeddings of K negatively sampled words.

The model maximizes similarity between true (context, target) pairs while minimizing similarity between context and randomly sampled negative words.

Two embedding matrices are learned:

- Input embedding matrix $W_{in} \in \mathbb{R}^{|V| \times d}$
- Output embedding matrix $W_{out} \in \mathbb{R}^{|V| \times d}$

After training, only the input embeddings are retained as the final word representations.

Preprocessing

The Brown corpus was preprocessed by:

- Converting all words to lowercase,
- Removing tokens containing non-alphabetic characters,
- Retaining only the top 10,000 most frequent words to limit vocabulary size.

Hyperparameter Selection

The following hyperparameters were used:

- **Vocabulary Size ($|V| = 20{,}000$):** Limiting the vocabulary to the 20,000 most frequent words reduces computational complexity while retaining the majority of meaningful tokens.And I tried to keep it comparable with svd because of which I chose 20,000 to tbe the vocabulary size
- **Context Window Size (2):** A symmetric window of size 2 (two words on each side) was used. This captures local syntactic and semantic information while avoiding excessive noise from distant words.

- **Embedding Dimension ($d = 100$):** An embedding size of 100 provides sufficient representational capacity to encode semantic relationships without overfitting or excessive computational cost.As said in the previous section , the data is not that large so large dimensions may cause overfitting.
- **Negative Samples ($K = 5$):** Five negative samples per positive pair were used. This provides an efficient approximation of the softmax objective while maintaining stable training dynamics.Lower number of negatives can provide faster training but it may result in weaker word representation.Higher number like 10 and 20 can give better embeddings for small datasets but with a cost of slower training, but this is not the main concern, it can cause the risk of Domination by negative, therefore I chose an inbetween value which is 5.
- **Negative Sampling Distribution ($f(w)^{0.75}$):** Words were sampled according to their frequency raised to the power of 0.75. This distribution balances frequent and rare words, preventing overly frequent tokens from dominating the sampling process, as suggested in the original Word2Vec work.
- **Batch Size (128):** A batch size of 128 provides stable gradient estimates while maintaining efficient GPU/CPU utilization.
- **Optimizer (Adam, learning rate = 0.001):** The Adam optimizer was chosen for its adaptive learning rate properties, which improve convergence speed and stability.
- **Number of Epochs (50):** The model was trained for 50 epochs to ensure sufficient exposure to the corpus while preventing overfitting.

1.2.1 Architecture

The CBOW model is implemented as a shallow neural network consisting of an input embedding layer and an output embedding layer. The model predicts a target word given its surrounding context words using negative sampling.

Input Layer Let the vocabulary size be $|V|$ and the embedding dimension be d. Each context word is represented by its index and mapped to a dense vector using the input embedding matrix:

$$E_{in} \in \mathbb{R}^{|V| \times d}$$

Given a context window of size m, the context words are:

$$[w_1, w_2, \ldots, w_m]$$

The embedding for each word is obtained as:

$$e_i = E_{in}(w_i)$$

Hidden Representation The embeddings of the context words are averaged to produce a single context vector:

$$h = \frac{1}{m} \sum_{i=1}^{m} e_i$$

where $h \in \mathbb{R}^d$ represents the context vector.

Output Layer The model contains another embedding matrix for output words:

$$E_{out} \in \mathbb{R}^{|V| \times d}$$

For the true target word w_t, the corresponding embedding is:

$$v_t = E_{out}(w_t)$$

The positive score between the context vector and the target embedding is computed using a dot product:

$$s_{pos} = h \cdot v_t$$

Sigmoid Activation The probability of the target word given the context is computed using the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$P(w_t | context) = \sigma(s_{pos})$$

Negative Sampling To avoid computing the full softmax over the vocabulary, the model samples K negative words $\{w_1^-, w_2^-, \ldots, w_K^-\}$ from a noise distribution.

For each negative word w_k^- with embedding v_k^-:

$$s_{neg}^{(k)} = h \cdot v_k^-$$

Loss Function The model maximizes the probability of the true target word while minimizing the probability of the negative samples. The loss function used is:

$$\mathcal{L} = -\log \sigma(s_{pos}) - \sum_{k=1}^{K} \log \sigma(-s_{neg}^{(k)})$$

where σ is the sigmoid activation function.

After training, the input embedding matrix E_{in} is used as the final word embedding representation.

Training Objective

The model minimizes the negative log-likelihood of the objective function above. Since the loss is averaged over mini-batches, the reported epoch loss corresponds to the mean loss per training instance.

Final Embeddings

After training, the learned input embedding matrix was saved and used as the final word embedding representation. These embeddings encode semantic similarity such that words appearing in similar contexts obtain similar vector representations.

2 Analysis: Are the Embeddings Fishy?

2.1 Analogy Test (Semantic Capability)

formula to be used: maximize $cos(x, vec(B) - vec(A) + vec(C)$

Analogies:

1. Paris : France :: Delhi : ? (Syntactic/Capital)
2. King : Man :: Queen : ? (Semantic/Gender)
3. Swim : Swimming :: Run : ? (Syntactic/Tense)

SVD

The following analogy results were obtained using embeddings generated via Singular Value Decomposition (SVD) of the co-occurrence matrix.

1. Paris : France :: Delhi : ? *(Capital–Country Relationship)*

- Expected: **India**
- Top Predictions:
  - securing (0.5219)
  - currents (0.5027)
  - differentiation (0.5011)
  - incidence (0.4897)
  - prevailing (0.4882)

2. King : Man :: Queen : ? *(Gender Relationship)*

- Expected: **Woman**
- Top Predictions:
  - way (0.8800)
  - himself (0.8660)
  - people (0.8644)
  - never (0.8615)
  - always (0.8558)

3. Swim : Swimming :: Run : ? *(Verb–Gerund Relationship)*

- Expected: **running**
- Top Predictions:
  - gone (0.8842)
  - coming (0.8826)
  - walk (0.8722)
  - running (0.8698)
  - turn (0.8687)

Discussion. The SVD-based embeddings fail to correctly solve the capital–country and gender analogies. Although the verb–gerund relation partially appears (*running* is present in the top predictions), it is not ranked first. This suggests that the SVD embeddings do not strongly encode linear semantic and syntactic regularities.

Word2Vec CBOW

The following analogy results were obtained using embeddings generated via Word2Vec (CBOW).

1. **Paris : France :: Delhi : ?** *(Capital–Country Relationship)*
   - Expected: **India**
   - Top Predictions:
     - indoors (0.4545853)
     - rituals (0.4159894)
     - oregon (0.40336722)
     - detergents (0.40153793)
     - rocks (0.39798656)

2. **King : Man :: Queen : ?** *(Gender Relationship)*
   - Expected: **Woman**
   - Top Predictions:
     - metrecal (0.38112772)
     - mountain (0.37779498)
     - match (0.36607707)
     - meeting (0.3658254)
     - battalion (0.36397997)

3. **Swim : Swimming :: Run : ?** *(Verb–Gerund Relationship)*
   - Expected: **running**
   - Top Predictions:
     - saddle (0.41394845)
     - liberal (0.3736282)
     - generating (0.36183512)
     - hunting (0.35875356)
     - sufficient (0.34280145)

Discussion. Same as the SVD the word2vec Cbow model didn't perform well,The Brown corpus just doesn't have enough training data for CBOW to develop the kind of precise geometric relationships needed for analogy tasks

Fast Text

The following analogy results were obtained using embeddings generated via Fasttext.

1. **Paris : France :: Delhi : ?** *(Capital–Country Relationship)*
   - Expected: **India**
   - Top Predictions:
     - india (0.58299565)
     - bihar (0.57737046)
     - gujarat (0.57699525)
     - rajasthan (0.5751757)
     - lahore (0.5726942)

2. King : Man :: Queen : ? *(Gender Relationship)*

- Expected: **Woman**
- Top Predictions:
  - woman (0.8033898)
  - lady (0.6801964)
  - strongwoman (0.65320647)
  - girl (0.64935243)
  - man–and (0.64720654)

3. Swim : Swimming :: Run : ? *(Verb–Gerund Relationship)*

- Expected: **running**
- Top Predictions:
  - running (0.73224974)
  - runnning (0.63935506)
  - runs (0.63057214)
  - ran (0.6225318)
  - runned (0.6003527)

Discussion. FastText clearly wins here. It nails all three analogies — "india," "woman," and "running" come out on top with strong cosine similarity scores. This is not surprising since FastText was trained on a very large corpus consisting of Wikipedia, the UMBC webbase corpus, and the statmt.org news dataset (about 16 billion tokens in total). Because of this massive and diverse training data, it captures broad semantic and syntactic relationships that a small corpus like Brown simply cannot provide.

2.2 Task 2: The Bias Check (Ethical Capability)

Cosine similarity can be found using:

$$\cos(\vec{w}_1, \vec{w}_2) = \frac{\vec{w}_1 \cdot \vec{w}_2}{\|\vec{w}_1\|\|\vec{w}_2\|} \tag{1}$$

I have evaluated the following pairs:

- Pair A: $\cos(\text{doctor}, \text{man})$ vs $\cos(\text{doctor}, \text{woman})$
- Pair B: $\cos(\text{nurse}, \text{man})$ vs $\cos(\text{nurse}, \text{woman})$
- Pair C: $\cos(\text{homemaker}, \text{man})$ vs $\cos(\text{homemaker}, \text{woman})$

GloVe Embeddings

Pair	**cos(., man)**	**cos(., woman)**	**Difference**
Doctor	0.4012	0.4691	0.0678
Nurse	0.2373	0.4496	0.2123
Homemaker	0.0529	0.2857	0.2328

Table 1: Cosine similarity scores for GloVe embeddings.

All three professions show higher similarity with *woman* than with *man*, indicating a measurable gender association in the embedding space. The strongest bias is observed for *homemaker*, followed by *nurse*.

Pair	cos(., man)	cos(., woman)	Difference
Doctor	0.5379	0.5738	0.0360
Nurse	0.4669	0.5718	0.1049
Homemaker	0.4211	0.5249	0.1038

Table 2: Cosine similarity scores for FastText embeddings.

FastText Embeddings

Similar to GloVe, FastText embeddings also show higher similarity with *woman* across all professions. However, the magnitude of the bias is smaller compared to GloVe, particularly for the word *doctor*.

Discussion

Both pre-trained embeddings encode gender associations. Professions such as *nurse* and *homemaker* exhibit stronger association with *woman*, reflecting common societal stereotypes present in the training data.

This demonstrates that word embeddings not only capture semantic relationships but also inherit social biases from their underlying corpora.

3 POS Tagging with Multi-Layer Perceptron

3.1 Model Architecture

1. Context Window (Input) The model considers a fixed-size context window around the target word. For a context size C (typically $C = 2$), the total input consists of $2C + 1$ words: the center word, C words to the left, and C words to the right.

2. Embedding Layer Each word index in the context window is mapped to a dense vector of dimension embedding_dim (e.g., 100). Special tokens such as `<PAD>` and `<UNK>` are handled appropriately (e.g., padding mapped to a zero vector). All word embeddings in the window are concatenated into a single flattened vector of size:

$$(2C + 1) \times \text{embedding_dim}$$

For example, $5 \times 100 = 500$ dimensions when $C = 2$ and embedding dimension is 100.

3. Hidden Layer 1 The flattened input is passed through a fully connected layer of 256 units, followed by a ReLU activation. Dropout is applied for regularization (e.g., 30% dropout).

4. Hidden Layer 2 The representation is further projected to 128 units, followed by another ReLU activation and Dropout layer.

5. Output Layer A final linear layer maps the hidden representation to num_tags outputs. Each output corresponds to the unnormalized score (logit) for a POS tag. Softmax is applied implicitly during training via the cross-entropy loss.

Finetunig or Freezing the embeddings I have chosen to fine-tune the embeddings along with the model instead of freezing them.

For pre-trained embeddings such as FastText and GloVe, fine-tuning may be somewhat redundant because they were trained on very large corpora and already provide strong semantic representations of words.

However, for embeddings trained on a smaller dataset—such as SVD-based embeddings or Word2Vec (CBOW) trained on the Brown corpus—the representations may not be sufficiently rich or generalizable. In this case, fine-tuning during training allows the embeddings to adapt to the specific task and dataset, thereby improving their quality and incorporating additional contextual information.

3.2 Hyperparameters

The following hyperparameters were used for training the POS tagging model:

Hyperparameter	**Value**
Context Window Size (C)	2
Batch Size	128
Learning Rate	0.001
Optimizer	Adam
Dropout Rate	0.3
Number of Epochs	20
Minimum Word Frequency	2
Loss Function	Cross Entropy Loss

Table 3: Hyperparameters used for training the POS tagging model.

3.3 Evaluation

Model	**Accuracy**	**Macro-F1**
Model with SVD Embeddings	0.9667	0.8974
Model with CBOW Embeddings	0.9748	0.9313
Model with fastText Embeddings	**0.9759**	**0.9333**

Table 4: Performance comparison of different embedding variants on the POS tagging task.

Confusoin matrix(fasttext - best model)

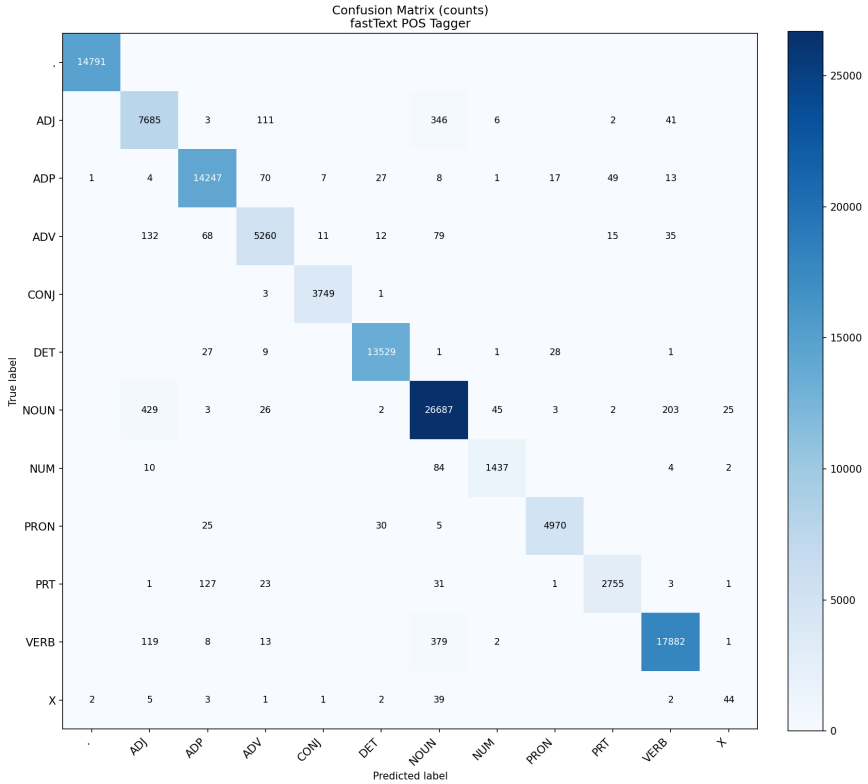

Figure 1: Confusion matrix for the POS tagger trained with FastText embeddings.

Note: The blank places represent the count 0

Table 5: POS Tagging Performance using FastText Embeddings

Tag	**Precision**	**Recall**	**F1-score**	**Support**
.	0.9998	1.0000	0.9999	14791
ADJ	0.9165	0.9379	0.9271	8194
ADP	0.9818	0.9864	0.9841	14444
ADV	0.9536	0.9373	0.9454	5612
CONJ	0.9950	0.9989	0.9969	3753
DET	0.9946	0.9951	0.9948	13596
NOUN	0.9649	0.9731	0.9690	27425
NUM	0.9631	0.9349	0.9488	1537
PRON	0.9902	0.9881	0.9892	5030
PRT	0.9759	0.9364	0.9558	2942
VERB	0.9834	0.9716	0.9775	18404
X	0.6027	0.4444	0.5116	99
Macro Avg	**0.9435**	**0.9253**	**0.9333**	**115827**

Overall Accuracy: 0.9759

4 Analysis and Report

4.1 Embedding Comparison

From the results of Section 2, the pre-trained embeddings (GloVe and fastText) clearly outperformed SVD and Word2Vec embeddings. The top-5 predicted words for the pre-trained models were semantically and syntactically meaningful, and closely aligned with the expected answers. This indicates that their embedding space effectively captures relational structure.

In contrast, embeddings trained on the smaller Brown corpus (SVD and Word2Vec) often produced top-5 predictions that were not strongly related to the expected analogy outputs. This suggests that high-quality semantic structure in embedding space generally requires training on large-scale corpora. Pre-trained embeddings benefit from exposure to vast amounts of linguistic data, allowing them to encode richer and more reliable relationships between words.

4.2 Hyper Parameters in MLP

1. Training Hyperparameters

- **Learning Rate (0.001):** The step size used by the Adam optimizer to update the model weights during training.
- **Batch Size (128):** The number of context windows processed together in a single forward and backward pass.
- **Optimizer:** I have used the Adam optimizer.
- **Number of Epochs (20):** The total number of complete passes over the entire training dataset.

2. Data Preprocessing Hyperparameters

- **Minimum Frequency (2):** The minimum number of occurrences required for a word to be included in the vocabulary. Words appearing fewer times are mapped to the `<UNK>` (unknown) token.

4.3 Error Analysis

Model: `pos_tagger_svd.pt`

Total Sentences: 57,340
Total Errors: 3,857
Approximate Accuracy: 0.9667

Example 1 (NUM → NOUN) **Sentence:** That was up from 173 per cent in the preceding 12 months.
Target Word: '173'
Context Window: up from 173 per cent
Analysis: The word was Out-of-Vocabulary (OOV) and mapped to `<UNK>`, forcing the model to rely only on context, leading to an incorrect NOUN prediction.

Example 2 (ADJ → NOUN) **Target Word:** 'grassroots-fueled'
Context Window: of this grassroots-fueled machine
Analysis: OOV word mapped to `<UNK>`, losing morphological cues and resulting in a NOUN prediction.

Example 3 (VERB → ADJ) **Target Word:** 'tired'
Context Window: sick and tired of hearing
Analysis: The word 'material' is highly ambiguous. Based on the training set, it can be assigned to the following tags: ['NOUN', 'ADJ']. The model incorrectly determined its role here as ADJ instead of NOUN, likely due to an overlapping context pattern.

Example 4 (ADJ → ADV) **Target Word:** 'longer'
Context Window: gives them longer holiday periods
Analysis: Ambiguous (ADJ/ADV); contextual similarity likely biased the prediction toward ADV.

Example 5 (ADV → NOUN) **Target Word:** 'tensely'
Context Window: colonel crouched tensely on one
Analysis: Seen only as ADV in training, but predicted NOUN, suggesting contextual overgeneralization or weak long-range dependency modeling.

Model: `pos_tagger_cbow.pt`

Total Sentences: 57,340
Total Errors: 2,814
Approximate Accuracy: 0.9757

Example 1 (VERB → NOUN) **Target Word:** 'lampoon'
Context Window: writers will lampoon baseball personalities
Analysis: OOV word mapped to `<UNK>` leading to incorrect NOUN prediction.

Example 2 (NOUN → VERB) **Target Word:** 't's'
Context Window: script – t's carefully crossed
Analysis: OOV token mapped to `<UNK>`, causing reliance on context and misclassification as VERB.

Example 3 (NOUN → ADJ) **Target Word:** 'material'
Context Window: spiritual and material stake in
Analysis: Ambiguous (NOUN/ADJ); contextual similarity led to incorrect ADJ prediction.

Example 4 (VERB → ADV) **Target Word:** 'sure'
Context Window: don't sure 'nuff me
Analysis: Ambiguous usage; model predicted ADV instead of VERB due to contextual overlap.

Example 5 (NUM → NOUN) **Target Word:** '1902'
Context Window: (1902) resembles
Analysis: Although seen as NUM in training, predicted NOUN, indicating contextual overgeneralization.

Model: `pos_tagger_fasttext.pt`

Total Sentences: 57,340
Total Errors: 2,791
Approximate Accuracy: 0.9759

Example 1 (X → ADJ) **Target Word:** 'arte's'
Context Window: the pro arte's contribution i
Analysis: OOV word mapped to `<UNK>`, resulting in incorrect ADJ prediction.

Example 2 (NOUN → VERB) **Target Word:** 'asimov's'
Context Window: isaac asimov's the caves
Analysis: OOV token mapped to `<UNK>`, causing misclassification as VERB.

Example 3 (NOUN → ADJ) **Target Word:** 'equivalent'
Context Window: child's age equivalent dots
Analysis: Ambiguous (NOUN/ADJ); contextual similarity led to ADJ prediction.

Example 4 (NOUN → ADJ) **Target Word:** 'mobile'
Context Window: of the mobile district was
Analysis: Ambiguous (NOUN/ADJ); misclassified due to contextual overlap.

Example 5 (ADJ → NOUN) **Target Word:** 'unplowed'
Context Window: unfenced, unplowed, unbroken
Analysis: Although seen only as ADJ during training, predicted NOUN, suggesting contextual bias or limited dependency modeling.

Google Drive Link: `https://drive.google.com/drive/folders/1X0F9B1r9o_z_AyzkcqSq9mS9VWUFL-pk?usp=sharing`